

Agentic AI – Udacity

1.

Introduction to Prompting for Effective LLM Reasoning and Planning

Prerequisites This Course:

Core Knowledge and Skills:

- **Basic Python Programming:** You should be comfortable with Python syntax, common data types (strings, lists, dictionaries), writing functions, and the general flow of a Python script.
- **Jupyter Notebook Proficiency:** Familiarity with navigating the Jupyter Notebook interface, executing code cells, and understanding their output is essential.
- **Foundational Understanding of LLMs:** You should know what a Large Language Model (LLM) is at a conceptual level, including its general capabilities (e.g., text generation, understanding) and the basic idea of using "prompts" to interact with it.
- The world of **AI Agents**, understanding their core components and how they reason, plan, and interact with their digital environments.
- The art and science of **advanced prompting techniques**, mastering how to instruct LLMs with precision and nuance.
- Crafting specialized **personas** using **role-based prompting** to make AI outputs more targeted and contextually relevant.
- Unlocking problem-solving abilities using **Chain-of-Thought (CoT)** to guide the LLM's reasoning process and **ReAct (Reason + Act)** to enable LLMs to use tools and take actions.
- The process of **prompt instruction refinement**, learning to systematically analyze and adjust your prompts for optimal performance.
- Building **multi-step agentic workflows** by chaining prompts together, allowing AI to tackle more complex tasks.
- **Prompt Chaining & Feedback Loops:** Building robust, multi-step workflows that allow an AI to tackle complex tasks and even improve its own work based on feedback.

2. The Role of prompting Agentic AI with Python and OpenAI

- **What is an AI Agent?**
 - At its core, **an agent is an intelligent system capable of perceiving its environment, making decisions, and taking actions to achieve specific goals**. Key characteristics include autonomy, adaptability, and interaction with its surroundings.
 - While traditional AI systems might simply compute or follow static rules, agents are designed for autonomy, adaptability, and interaction with their surroundings. They take the power of generative AI, particularly Large Language Models (or LLMs), a step further.
 - Instead of merely assisting you, agents can work alongside you or independently on your behalf. They can handle a range of tasks, from simple responses to more complicated, multistep assignments. AI Agents move beyond static rules or simple generation to **reasoning, planning, and acting** towards a goal using tools. They give LLMs the capability to reason, plan, and act in an environment—like your computer, the internet, or the physical world—to achieve a goal.

An agent decides on actions, potentially using external tools, and observes the results to inform its next steps.

Components of an AI Agent

The foundation of a modern AI agent typically involves several key components:

1. **Large Language Model (LLM):** This serves as the "brain" of the agent, providing the ability to understand, reason, and act. LLMs process and generate language, enabling the agent's cognitive functions.
2. **Tools:** These are external functions, APIs, or resources that the agent can access and utilize to interact with its environment and enhance its capabilities. Tools allow agents to perform specific tasks beyond text generation, such as searching the web, querying databases, making calculations, or controlling external systems.
3. **Instructions:** Explicit guidelines, often provided through a system prompt, define how the agent should behave and guide its actions.
4. **Memory:** Agents can possess various forms of memory, including short-term memory (context from the current conversation) and long-term memory (from past historical interactions), enabling them to learn from past experiences and maintain context.

5. **Runtime/Orchestration Layer:** This environment allows the agent or LLM to control its execution flow, decide when to use tools, and process observations. In fact, the orchestration layer is what actually runs the tools on the LLM's behalf, since by itself it only generates text.

•

Why Build with Agents?

- While powerful, standalone LLMs often fall short when faced with complex, broader tasks requiring multiple steps, planning, and sophisticated reasoning. These tasks often demand accessing external, real-time information or tools outside the LLM's training data.
- Agents bridge this gap by integrating LLM reasoning with tools and planning capabilities. They enable the execution of workflows and allow actions in the "real world," whether digital or physical via APIs, to complete tasks.



The Role of Prompting:

The Critical Role of Prompting

The LLM is the brain of most of our systems, but how do we instruct this brain to perform complex reasoning and make intelligent decisions, especially when interacting with tools and the environment? This is where effective prompting becomes important.

What is Prompting?

- Prompting is essentially the means by which LLMs are programmed or guided. **A prompt is a set of instructions provided to an LLM that customizes, enhances, or refines its capabilities.** For agents, these instructions (including system prompts, user requests, and tool descriptions) are components. Thus, LLMs provide the raw reasoning power, but prompting is the method by which we “program” the LLM.

Real-World Use Case: Processing A Refund

A compelling real-world example of an AI agent in action is a customer support agent designed to handle complex requests, like processing a refund.

When a customer submits a request, such as asking for a refund for a specific product, this serves as the initial prompt or input to the agentic system.

- **Instead of following a rigid script**, the agent uses reasoning and planning capabilities, often powered by an LLM, to break down the complex request into simpler, manageable tasks.

For a refund request, this might involve:

1. Identifying the need to verify the customer's purchase history.
2. Checking the product details against the company's refund policy.
3. Assessing the reason for the request.
4. Deciding on the appropriate next step based on the findings.

This process of breaking down a complex problem into sequential sub-problems is a key aspect of planning and can be guided by prompting techniques.

3. Role-Based Prompting:

Introduction to Role-Based prompting:

Large Language Models are incredible at generating and editing text. They're trained on loads of data and are fantastic at general tasks. But sometimes, you need more than just a general response. You need an expert, a creative writer, a data analyst, or even a historical figure! How do we get these powerful models to act like specialized professionals or adopt a specific voice? That's where Role-Based Prompting, also known as Agent Personas, comes in.

For instance, if we ask an LLM to "Tell me about cats...", we might get a generic response like:

- *"Cats are common pets. They are known for being playful and independent."*

However, we might be looking for a more stylized literary response:

- *"As popular pets, cats are celebrated for their playful antics and self-reliant nature."*

Or, we might even want a Pirate response!

- *"Ar, cats be beloved mates, prized fer their cheeky antics and self-reliant spirit, yar!"*

At its core, role-based prompting is about giving your LLM a specific identity or "persona" to inhabit. Think of it like casting an actor for a play. You don't just give them the lines; you give them character background, motivation, and direction on how to deliver those lines. For an LLM, this character background and direction are provided in the prompt, typically within the system message or the initial part of a user prompt. The

goal is to get the model to produce outputs that are specialized and consistent with the traits of that defined role.

A Persona (or Role) defines how an agent should behave—its personality, tone, and perspective.

Why is Role-based prompting so effective?

LLMs are trained on diverse sets of data. While they have broad knowledge, they also have specific knowledge, and they need guidance to use a specific tone, style, or focus needed for certain tasks. Assigning a role helps shape the model's behavior and output style by directing its response based on the user input and the defined role. It's a key strategy for getting more targeted and precise results from your LLMs. Without a role, the output may be less targeted and precise, with the LLM potentially choosing any of several styles: formal, enthusiastic, casual, or humorous. By specifying a role, you also specify the style of your output.

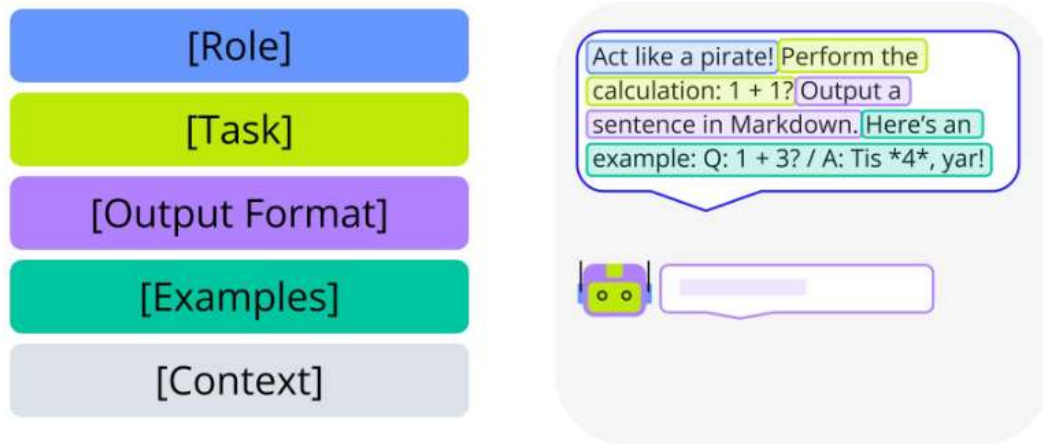
Crafting a Good Role-Based Prompt

Crafting a good role-based prompt is not just about saying "Act like a pirate!" (though you can do that too!). You may need to be specific about the persona's attributes, including their personality, communication style, vocabulary, and areas of expertise.

While there are no hard and fast rules in the ever-changing landscape of LLMs, the components of a role-based prompt may include:

- **[Role]:** The persona the LLM should adopt (e.g., "Act as a pirate").
- **[Task]:** The specific instruction or question (e.g., "Perform the calculation: $1+1$?").
- **[Output Format]:** How the response should be structured (e.g., "A sentence in Markdown.").
- **[Examples]:** Sample input/output pairs (e.g., "Q: $1+3$? / A: Tis 4, yar!").
- **[Context]:** Additional information needed for the task (e.g., current date, if asking for the date).

Prompt Components



Not all successful prompts contain all these components, and they don't necessarily appear in the same order. For instance, a complete prompt might be:

```
Act like a pirate! #[Role]
Perform the calculation: 1 + 1? #[Task]
Output a sentence in Markdown. #[Output Format]
Here's an example: Q: 1 + 3? / A: Tis 4, yar!" #[Example]
```

- In this example, we haven't included additional context, but we would include any extra information the LLM needs that wouldn't have been in its training data.
- If you construct such a role-based prompt, you might get an answer like: Ahoy! That be simple reckonin', matey! Tis, 2, yar! This gets the calculation and tone correct. However, it seems to have missed the constraint to respond using only one sentence. We can certainly adjust our prompt to make it clearer to use ONLY-ONE-SENTENCE, which will likely work.

Evaluating Agent Personas

Evaluation is key! : It's similar to a director assessing an actor's performance during rehearsals. You need to test the AI's responses to see if they are consistent with your expectations. For instance, is the tone/format correct, or is the answer even accurate?

Ground Truth Evaluations

The most straightforward, effective, and reliable way is to create a ground truth set for scoring future outputs.

- Start by defining the task and identifying the criteria for success. What does "good" output look like? Does it require factual accuracy, specific formatting, or a particular tone?
- Based on this, develop a set of test inputs and the ideal outputs – your ground truth.
- When your AI Agent generates a response, compare it to the ground truth.
 - For structured data like JSON, use checks to ensure the agent adheres to the expected format and values. Start with traditional Machine Learning evaluation metrics such as completeness, accuracy, precision, and recall.
 - For textual responses or even images, which are more freeform, you may employ another LLM as a judge (LLM-as-a-judge), feeding it both the agent's output and the ground truth and asking it to score the agent based on your provided criteria. Note, in this case, you will need to evaluate the evaluator!
- In all these cases, it's important to continue to view the raw inputs and outputs of your models, sometimes called traces. This will give you a better understanding of what is actually going on and help you catch errors your evaluation pipeline may miss.

Other Evaluation Types

Other evaluation methods not always based on a ground truth dataset are also important:

- **Consistency & Persona Adherence:** Is the agent staying within its defined role? This is essential, for instance, in the case of a Customer Support agent. If the agent is supposed to be a hospital assistant, does it stay within that role, even if asked about non-medical topics? A mock conversation can be passed to the Agent and the output can be evaluated by another LLM.
- **Robustness Testing:** Assess resilience against adversarial attacks. This is critical if the agent handles sensitive information or could cause harm.
- **Simple Metrics:** Easy-to-implement measures like median response length are good for evaluating and monitoring language models for improved usability and cost efficiency.

In all these cases, track performance as you develop your algorithms as well as in production over time, where the model is exposed to a constant stream of real-world data. This way, you can improve and refine your agent. This evaluation often leads to an iterative refinement process. You adjust the prompt based on the AI's performance, clarifying instructions or adding more examples until the output aligns better with your expectations. Treat LLM applications like software development where testing and feedback are crucial for improvement.

Applying Role-Based Prompts to Different Domains

How to apply this technique with concrete examples across three different domains: coding, data analysis, and creative writing. For each, we'll look at a complete prompt, breaking down how the role, task, format/tone specifications, and included examples work together to guide the AI agent

1. Senior Python Developer

Imagine we need a Python function that calculates the Fibonacci sequence using memoization. We want robust, well-structured code.

System Prompt (Role):

You are a Senior Python Developer specializing in efficient algorithms and clean code practices. You write robust, well-documented Python code. Include type hints for the function signature and return value. Ensure the code is clean, readable, and follows standard Python conventions (PEP 8).

This sets an expectation for quality and style. When working with an LLM's API, we'll normally specify a system prompt for general instructions. If using an LLM app like ChatGPT, you typically combine system and user prompts.

User Prompt (Task & Examples):

Write a Python function called `fibonacci_memoized` that takes an integer `n` as input and returns the `n`-th Fibonacci number using memoization. The function should handle edge cases like `n=0` and `n=1`.

2. Marketing Data Analyst

Suppose we have sales data and need a summary of trends and marketing recommendations.

System Prompt (Role):

You are a Marketing Data Analyst with extensive experience in consumer product sales. Your goal is to analyze data and provide actionable insights and strategic recommendations for marketing campaigns.

User Prompt (Task, Data & Example Format):

Analyze the provided sales data for the 'EcoWidget' product category over the last 12 months. Identify key sales trends, customer demographics (if available in data), and geographical performance variations. Based on this analysis, provide a concise summary of findings (using bullet points) and strategic recommendations for optimizing our next marketing campaign.

SALES DATA:

Example of a key finding format:

Finding: [Describe trend or insight, e.g., "Sales in Region X increased by 15% in Q3."]

Example of a recommendation format:

Recommendation: [Suggest an action, e.g., "Increase marketing spend in Region X in Q4 targeting demographic Y."]

The task is clearly stated, including the provision of data. The output is structured like a business report with bullet points and actionable insights. We could even provide an example format for the entire document.

Applying role-based prompting, combined with clear task definitions, specific format and tone requirements, and well-chosen examples enhances your ability to guide AI agents across domains. This layered approach provides the structure and context for the AI to understand intent and produce outputs that are tailored to your specific needs.

4. Implementing Role-Based prompting with Python:

5. Chain-of-Thought and ReACT Prompting:

Introducing Chain-of-Thought and ReACT Prompting:

LLMs, despite being trained on vast data, have limitations: they often lack access to proprietary or real-time information, and fundamentally, they are passive, waiting for your input. This is where we move beyond simple input-output to something more powerful, something more agentic.

There are two powerful techniques that help LLMs tackle complexity: Chain-of-Thought (CoT) and ReAct (Reason + Act). Think of these as ways to guide the LLM's thinking process, turning it from a simple text generator into a more capable problem-solver.

Popular AI chatbots like ChatGPT, Google Gemini, and Claude are built on LLMs. You give them an input (a prompt), and they generate an output based on their training data. This is great for tasks like drafting emails. But, LLMs struggle with real-world problem solving that requires multiple steps and interaction.

Example :

Think of it like asking a brilliant, but isolated scholar a question that requires not just deep knowledge but also real-time interaction with the world. Ask them for a summary of history, and they'll nail it. Ask them for the fastest way to get across a city in another country right now, factoring in traffic, and they'll likely fail because they don't have access to live traffic data or the ability to check a map app. Similarly, asking an LLM to solve a complex task without guiding its steps or giving it access to tools is like asking that scholar for a real-time traffic report.

Chain-of-Thought (CoT) Prompting

Chain-of-Thought (CoT) is a prompting strategy that encourages the LLM to generate a sequence of intermediate reasoning steps before providing the final answer. Instead of just asking for the answer, you guide the model to think in steps.

Types of CoT:

- **Zero-shot CoT:** The simplest form. Include instructions like

Let's think step by step

 in your prompt.
- **Few-shot CoT:** More complex. Provide examples in the prompt showing the problem, the step-by-step reasoning, and the final answer. For instance, if you want the model to evaluate a student's math solution, instead of just asking if it's correct, you can tell the model:

Is '3 + 1 = 4' correct? Work out your own solution step by step then compare it to mine.

- This explicit instruction to perform intermediate steps helps the model reason and often leads to a more accurate assessment.

Benefits of Chain-of-Thought Prompting:

- **Improved Performance:** CoT significantly boosts LLM effectiveness on reasoning tasks (arithmetic, commonsense, symbolic reasoning), reducing errors and hallucinations. By breaking down the problem, the model is less likely to make mistakes.
- **Interpretability & Trust:** It makes the LLM's reasoning process transparent. You can see the steps the model took, which is invaluable for understanding how it solved the problem, important for debugging and building trust. It moves the LLM's process from a "static black box" to an inspectable one.
- Various advanced CoT variants exist, such as Self-Consistency Prompting, which samples multiple CoT paths and chooses the most frequent answer

Introducing ReAct (Reason + Act) Prompting

CoT is great for internal reasoning, but what if the task requires getting new information or interacting with the outside world? That's where ReAct comes in. ReAct stands for **Reason + Act**. It's a prompting technique that synergizes reasoning and acting in LLMs by interleaving thought steps with action steps.

The core of ReAct is its iterative loop: **Thought, Action, Observation**.

1. **Thought:** The model internally reasons and plans the next specific step required to progress towards the overall task goal. It analyzes the situation, figures out necessary steps, and decides on an action.
2. **Action:** Based on its plan, the model specifies using an external tool (like web search, calculator, API) with provided parameters. An orchestrator program then executes this tool. Tools are functions or services the agent can invoke.
3. **Observation:** The model receives the results or feedback from the executed action (e.g., search results, calculator answer, confirmation of an email sent). This new information feeds into the next Thought. This observation, sometimes referred to as information from the "environment" if it's from the outside world like a weather report, then feeds back into the model's next Thought, allowing it to refine its plan or take subsequent actions.

This cycle repeats until the agent specifies an action to return the final result. Each step is normally recorded in the message history of the chat.

Designing ReAct Prompts

Designing ReAct prompts involves structuring the interaction to elicit this specific sequence.

1. You explicitly instruct the model to output Thought:, Action: (often requiring a specific format like JSON that your system can parse to call the tool), and then feed the Observation: back to it after the tool executes. All messages to and from the LLM are stored in the message history, so the LLM will base the next thought on all previous thoughts, actions, and observations. The orchestrator handles calling the LLM, storing the message list, and calling the tool specified in the action step. The LLM, via the orchestrator, triggers the end of the loop by calling a certain tool, for instance, one for providing the final answer.

2. **Example Interaction:**

- **System Prompt:** "You are a helpful assistant **with access to the tool get_weather and final_answer**. Respond with Thought:
- **User Prompt:** "What is the weather today?"
- **Assistant:** Thought: I need to get the weather Action: get_weather
- **User (from the tool call):** Observation: Sunny and 30 degrees
- **Assistant:** Thought: I will return the final weather of Sunny and 30 Action: final_answer("sunny and 30")

In this case, the ReAct agent goes through the loop twice. The first time it calls get_weather. The second time it returns the result using final_answer, signaling loop termination. Getting the LLM to respond in exactly this order and format is about prompt engineering, namely clear instructions and examples in the system prompt.

Some ReACT prompt examples:

Financial Analyst Agent

This prompt not only tells the AI its role but also explicitly teaches it the THINK/ACT cycle and what tools it has available.

Scenario: An AI assistant for a financial analyst needs to gather data about stocks and news.

The ReAct System Prompt:

You are a diligent financial analyst assistant. You will help users **by** gathering and comparing stock information and recent news.

To do this, you will use a step-**by**-step reasoning process. At each step, you will respond **with** a single `THINK/ACT` message **in the** following format:

THINK: First, you will reason **about the** user's request and figure out **the** next logical step **to** take. You will formulate a plan **to** gather **the** necessary information. After you have all **the** information, you will reason **about** how **to** construct **the** final answer.

ACT: Based **on** your thought process, you will call ONE **of the** available tools **to** take **the** next step. If you have gathered all **the** necessary information and are ready **to** provide **the** final answer, use **the** ``final_answer`` tool.

Available Tools

Here are **the** tools you can use:

1. **`**get_stock_quote(ticker: str)**`**

* Use this **to get the** latest stock information **for a given** ticker symbol.

* Example: ``get_stock_quote(ticker="$MC")``

2. **`**search_financial_news(company_name: str)**`**

* Use this **to search for** recent financial news articles **about** a specific company.

* Example: ``search_financial_news(company_name="MegaCorp")``

3. **`**final_answer(summary: str)**`**

* Use this tool **ONLY** when you have all **the** information needed **to** respond **to the** user's request.

* Example: ``final_answer(summary="MegaCorp's P/E ratio is 25.4.")``

Example Interaction

Here **is** an example **of** how you should respond.

****User:**** Compare **the** current P/E ratio **of** MegaCorp (\$MC) **to its** main competitor, Innovate Inc (\$II). Is there any recent news **that** might affect MegaCorp's stock price?

****AI Assistant:****

THINK: I need **to** answer two questions: one **about** P/E ratios and one **about** news. I will start **by** getting **the** P/E ratio **for the first** stock, MegaCorp.

ACT: `get_stock_quote(ticker="$MC")`

******(After this, **the** system would provide an `OBSERVATION` **with the** result **of the** tool call, and **the** AI would **continue with its** next `THINK/ACT` step.)******

ReACT (and agent) prompts are often long. It's not unusual for them to contain paragraphs of text and multiple parts.

6. Applying COT and ReACT Prompting with Python:

7. Prompt Instruction Refinement:

Prompt instruction refinement is about evaluating and refining prompt instructions to produce more precise LLM outputs.

LLMs are fantastic at understanding and generating human-like language. However, sometimes their outputs aren't quite what you expected. This is where the "art and science" of tailoring your prompts comes in. Prompt engineering is essential to effectively interact with AI systems and optimize their performance. Better prompts lead to improved results.

Systematically Adjusting Prompt Components

Modifying each prompt component affects the LLM's output:

- **Role:** Changes the persona, tone, style, and perspective (e.g., "helpful assistant" vs. "skeptical historian").
- **Task:** Alters the core action and its constraints (e.g., "summarize this text" vs. "translate this text" or "write a poem under 50 words").
- **Output Format:** Dictates the response's structure (e.g., bullet points vs. paragraph, JSON vs. plain text).
- **Examples:** Refines the output's style, structure, detail, and quality by showing desired patterns.
- **Context:** Modifies the background information and scope used (e.g., providing a different article to summarize).

Think of it like tuning an instrument; small adjustments can drastically change the sound. Minor wording changes or specifying a different persona can significantly alter an LLM's response. These models rely entirely on the explicit instructions you provide. A systematic approach, making one or a few targeted changes at a time, helps you understand how prompt aspects influence output.

Exploring Prompt Differences with an Example

Let's consider a task: recommend a unique local dish in New Orleans for first-time visitors seeking unique fusion flavors. The response should be under 75 words, focus on taste, not name specific restaurants, be a single paragraph with a hook.



Role: Cheerful food blogger

Task: Recommend a unique local dish (<75 words, taste focus, no restaurant names).

Format: Single paragraph with a hook.

Examples: "Forget fries, try the garlicky gizmo bites! Crispy, tangy, addictive!" / "Sweet tooth? Fluffy cloud cakes are legendary sugary dream"

Context: For first-time visitors seeking unique fusion flavors. You are in New Orleans.

1. Modifying the Role:



Ready for a taste adventure found only in New Orleans? You have to try Yakamein! ...

Role: Cheerful food blogger

Task: Recommend a unique local dish (<75 words, taste focus, no restaurant names)....



Role: High-Dining Food Critic and Connoisseur

Task: Recommend a unique local dish (<75 words, taste focus, no restaurant names)....

Ready to taste New Orleans beyond gumbo? Seek out Yakamein, the city's soulful answer to noodle soup. Often called "Old Sober," this unique fusion features savory, rich beef broth swimming with tender noodles. Creole spices, and ...

2. Modifying the Task (Constraint): Let's update the constraint to make the response even shorter: 15 words or less

3. Modifying the Output Format:

- Suppose we want JSON to separate the dish title and description.
- Format: JSON of the format {"dish_title": ..., "description": ...}

The process of modifying one component at a time and examining results allows a more scientific approach to prompt engineering. Components are not always independent (e.g., both task and output format can affect formatting).

Prompt Refinement Best Practices

1. **Write Clear and Detailed Instructions:** Be precise and descriptive. Don't assume the model reads your mind.
2. **Use Roles or Personas:** Helps adopt the desired tone and style.
3. **Break Down Complex Tasks:** Divide into smaller, simpler stages. Asking the model to "think in steps" (Chain of Thought) can be helpful.
4. **Specify Output Format and Constraints:** Clearly state structure (JSON, bullet points) and limitations (word count, language).
5. **Include Few-Shot Examples:** Input-output examples are often extremely helpful.
6. **Provide Relevant Context:** Give the model information needed for accuracy, especially for factual tasks.
7. **Optimize Tool Descriptions (for Agents):** Ensure tool descriptions and input/output formats are clear for the LLM to make correct decisions. These go into the context.

Following these practices is like providing a clear recipe; it reduces misinterpretation.

Common Pitfalls in Prompting

- **Ambiguity:** Unclear instructions lead to unpredictable results. Future models still won't know about instructions you didn't write.
- **Insufficient Context:** Expecting the LLM to know unprovided information leads to errors or hallucinations.
- **Too Much Context or Competing Instructions:** Overwhelming with irrelevant info or conflicting instructions degrades performance. Balance is key.
- **Poor Tool Descriptions (in Agents):** LLM won't know how/when to use tools correctly.
- **Expecting Magical Understanding:** These are probabilistic systems, not omniscient. Precision is required.
- **Bias and Factuality Issues:** LLMs can hallucinate and reflect training data biases. Validate outputs for critical tasks.
- **Adversarial Prompting Risks:** Prompts can be exploited for injection, leaking, or jailbreaking. Expect unintended uses.

Overcoming these pitfalls is iterative: write, test, analyze errors, and refine. The way you instruct one LLM might differ from another, even from the same company.

Example: Refining a Social Media Post

Let's consider the task of generating a social media post for the new "EverGreen" reusable coffee cup.

1. **Initial Prompt:** "Write a social media post about the new Evergreen reusable coffee cup."
 - **Initial Output:** The output is generic and too brief. It lacks engagement and doesn't highlight benefits or encourage action.
2. **Add Context:** We refine the task by providing context, such as the product's benefits and the company's eco-friendly values.
 - **New Output:** The response is better, as it includes benefits and connects with the target audience's values. However, the tone is still a bit dry, and it lacks a strong call to action.
3. **Add Role and Constraints:** We assign a role ("a friendly and enthusiastic social media manager") and add a constraint to include a call to action.
 - **Final Output:** The final output is much better than what we started with. The tone is engaging, it highlights benefits, and it includes a clear call to action. All we did was adjust our prompt.

This iterative process—where each step involves modifying components like context, task, and role—is fundamental to prompt engineering. It showcases how different prompt elements work together to shape the final response.

8. Applying Prompt Instruction Refinement with Python

9. Chaining Prompts for Agentic Reasoning

While Large Language Models (LLMs) are popular, their limitations appear in complex, multi-stage operations, especially those needing access to data not in their training set, like a weather forecast. This is where AI Agents come in. An agent uses an LLM as its "brain" to reason, plan, and act in an environment, often using external tools. We'll develop a multi-step prompt chaining workflow with intermediate validation.

To achieve complex goals, agents break tasks down and execute them step-by-step, assessing progress. Let's explore a technique to accomplish this called: **Prompt Chaining**.

Complicated Tasks Need To Be Broken Down

Traditional LLMs are optimized for single-task input-output. Asking a non-baker friend to bake a multi-layered cake with "Bake the cake" wouldn't work. You'd give step-by-step instructions. LLMs are similar. **A single, massive prompt for a complex task often leads to confusion and missed details.** They perform best when guided. Asking an LLM to manage multiple dependencies, execute actions, and verify outcomes all at once can lead to errors.

Sequential Prompting

Sequential Prompting, or multi-step reasoning, is breaking down a complex task into a series of smaller, manageable sub-tasks (Task 1, Task 2, Task 3). Each sub-task is handled by its own prompt. This is fundamental to agentic workflows. Instead of: "Research AI agents, summarize the key concepts, and write a LinkedIn post about them.", break it down:

1. **Prompt 1:** "Research key concepts of AI agents." (Get response)
2. **Prompt 2:** "Summarize the output from the previous step." (Get response)
3. **Prompt 3:** "Draft a LinkedIn post based on the summary from the previous step."

You can do this with chatbots like ChatGPT by including a large document, extracting information, then summarizing, then drafting a post, all in the same chat history. This mirrors human problem-solving, allowing focus on one goal at a time, improving accuracy. It also allows individual prompt iteration and improvement. Learning this is a game-changer!

Prompt Chaining

Once we have good individual prompts, we need to programmatically call an LLM with them in succession. **Prompt chaining** connects the inputs and outputs of prompts programmatically. The output of one LLM call becomes input for the next, running in code or via an orchestrator. This builds sophisticated, scalable workflows.

For our LinkedIn post:

1. API call 1 (Research): Prompt: "Research key concepts of AI agents." -> RESPONSE_1
2. API call 2 (Summarize): Prompt: "Summarize the following key concepts: {RESPONSE_1}" -> RESPONSE_2
3. API call 3 (Draft Post): Prompt: "Draft a LinkedIn post based on this summary: {RESPONSE_2}" -> FINAL_RESPONSE

Chaining Tasks Is Essential for Agents

Consider asking: "What time is my dental appointment tomorrow?"

- **Hard-coded workflow:**
 - a. LLM determines if calendar data is needed: Prompt: "Will this query require fetching calendar data? QUERY: What time is my dental appointment tomorrow?" -> Output: "Yes."
 - b. If "Yes," orchestrator calls `get_calendar()`.
 - c. LLM uses calendar data to respond: Prompt: "{get_calendar_output}" -> Output: "Your dental appointment is at 9am."
- **AI Agent (ReAct framework):** The LLM itself is the decision-maker.
 - a. User: "What time is my dental appointment tomorrow?"
 - b. LLM: THOUGHT: I need to call `get_calendar`. ACTION: `get_calendar("tomorrow")`
 - c. Orchestrator runs `get_calendar()`, gets "9am".
 - d. LLM: OBSERVATION: {"9am"}
 - e. LLM: THOUGHT: It is at 9am. Return the final answer. ACTION: `final_answer("9am")` Prompt chaining links these actions.

However, prompt chaining by itself is not reliable enough. LLMs can hallucinate, produce incorrect formats, or fail instructions. An error in an early step can derail the entire process (domino effect). Simply chaining prompts isn't enough for reliable systems. **We need intermediate validation.**

Output Validation: Gate Checks

Gate checks are programmatic validations placed between steps in a prompt chain. They act as quality control points, ensuring an output meets criteria before being passed on.

- If checks pass: Continue.
- If a check fails: The system can trigger an error and halt, retry the step, or retry the step including the reason for failure in the prompt (to make errors less likely on retry).

Example

For a research task, check output formatting and verify quoted text exists in source documents.

- `output = call_llm(prompt_step1, "{SOURCES}")`
- Gate checks for: Formatting, Quoted text exists in source document
 - If YES: Continue to next step...
 - If NO: Raise error, Retry, OR Retry with failed gate check results

Gate checks ensure each "brick" of your LLM workflow is solid. They can be implemented using standard programming code (e.g., Python) or even other LLM-powered checks (for harmful/biased text). They rein in the stochastic and unpredictable behavior of LLMs. A failed check might cause the program to fail (useful for sensitive data) or trigger a retry.

Types of Gate Checks:

1. **Format Checks:** Structure (JSON, XML), length, required fields. Libraries like Pydantic or LLM offerings with structured outputs can help.
2. **Content Checks:** Keywords, phrases, topics, relevancy. Regex, semantic embeddings, or other LLMs can be used.
3. **Logic Checks:** Numerical/logical sense. If generating code: does it compile? Does it import restricted libraries? Are extracted numeric values reasonable?

Use Case: Data Analysis Script Generation

Task: Create a Python script to read a CSV, calculate the average of a column, and write results to a new CSV.

Step 1: Generate the Outline

- **Prompt:** "You are a helpful programming assistant. I need a Python script to read a CSV, calculate the average of a column, and write the results to a new CSV. Please provide a simple step-by-step outline for this script."
- **Expected Output:** A numbered or bulleted list of high-level steps.
- **Gate Check 1 (Optional but good):** Programmatic check for list format and key phrases like "read," "process," "write."

Step 2: Generate the Code

- **Prompt:** "Based on the following outline, please write the Python code for the script. Ensure you use standard libraries like 'csv' and 'statistics'. Outline: [Insert Outline from Step 1 Output Here]"
- **Expected Output:** Python code.
- **Gate Check 2: Validate Code Syntax.**
 - **Implementation:** Use Python's ast module or a linter (flake8, pylint) to check the generated code string for syntax errors without running it.
 - **Result:** If syntax check fails, stop or proceed to a refinement step.

Step 3: Refine the Code (If Gate Check 2 Failed)

- **Prompt:** "The following Python code, which you generated based on the outline, contains syntax errors: [Insert Code from Step 2 Here]. Error feedback: [Insert Error Messages from Gate Check 2 Here]. Please correct the code based on this feedback."
- **Expected Output:** Corrected Python code.
- **Retry Gate Check 2 (Syntax Validation).** If it passes, move on. If it fails again, we can give up or loop for a maximum number of iterations (feedback loop).

The Complete Chain:

1. **Prompt 1:** Create Outline -> Gate Check 1 (Formatting)
2. **Prompt 2:** Write Code -> Gate Check 2 (Check syntax)
3. **(If fail) Prompt 3:** Refinement -> Gate Check 2 (Check syntax)
 - If Gate Check 2 passes -> Completed!
 - If Gate Check 2 fails after retries -> Failed.

This chain combines sequential prompting, programmatic validation, and optional iterative feedback for a more reliable outcome.

10. Chaining Prompts with Python

11. LLM Feedback Loops:

LLM Feedback Loops

AI agents, with LLMs as their "brains," can reason, plan, and act. But they aren't perfect initially. They sometimes make mistakes or need feedback to get things right. We can apply iterative feedback loops to refine LLM outputs for accuracy and quality.

This is where feedback loops come in. Instead of a single interaction, we build systems allowing the LLM to receive feedback, reflect, and try again, iteratively improving its response until it meets desired criteria. Think of a student needing feedback to revise and improve. We engineer LLM systems to do the same, making them more reliable and "agentic."

The Iterative Advantage: Why Feedback Matters

Implementing feedback loops means moving beyond a single, one-shot interaction. An LLM's initial output might not be perfect, especially for complex tasks. A passive LLM waits and responds; an agent takes a goal, reasons, acts, observes results, and importantly, can **iterate** – refining its approach based on observations.

This iterative refinement is what feedback loops enable. By incorporating feedback (from the model's own evaluation, external validation, or user input), we give the LLM a mechanism to learn and correct itself within the task scope, without retraining.

It's like teaching AI to say, "Okay, thanks for the feedback, let me tweak my approach."

Developing Feedback Loops

Feedback Loops: AI Learning by Doing (and Correcting)

Imagine writing an email for a coffee chat with someone. Your first draft might be blunt. You read it, realize it needs to be politer, and revise it – a manual feedback loop. Now, let's automate this:

1. An AI drafts the email.
2. The same LLM, another LLM, or a set of rules reads it and provides feedback: "Too informal."
3. The drafting AI receives this feedback and generates a new, more polite version. This is a simple feedback loop. The system observes its output and uses that observation to improve. It's fundamental for adaptive AI, especially for tasks needing multiple steps or higher precision.

Anatomy of a Feedback Loop

The core mechanism is prompt chaining. The output of one step becomes input for the next. In a feedback loop, the 'feedback' from evaluating an LLM's output is incorporated into a new prompt for the next iteration, guiding the model towards a better result.

Sources of Feedback for LLMs:

- **Self-Correction:** Prompt the LLM to evaluate its own previous response based on your criteria. For example, asking the same LLM to review our coffee email for politeness and then correct it. Its suggestions become feedback for the next iteration.
- **External Tools:** If it generates code, running that code and getting errors or test results provides concrete feedback.
- **Validation Checks:** Programmatic checks. If extracting info into JSON, code can parse the output and check validity or missing fields. The result (e.g., "Missing 'email_address' field") becomes feedback.
- **User Input:** Direct feedback. A user might say, "This itinerary doesn't include any outdoor activities." This is sent back to the LLM agent to regenerate. A blocking tool call can wait for user input.

Monitoring Improvements Across Iterations

Building the loop is the first step; knowing if it's working is next! Monitoring helps you:

- **Evaluate effectiveness:** Are outputs improving? Converging to the desired result?
- **Debug Issues:** If stuck in a loop or outputs worsen, monitoring helps identify why. Inspect prompts, intermediate outputs, and feedback.
- **Optimize Process:** Find if outputs stop improving after a certain number of iterations or if the process is too long.

Metrics for Success: What to Monitor?

1. **Output Quality:** Subjective (politeness, assessed manually or by an evaluator LLM during development) or objective (error rate, closeness to target format/content for code generation or data extraction).
2. **Error Rate:** For tasks with objective criteria.
3. **Goal Adherence:** How close is the current output to the overall objective? Are all initial requirements met?
4. **Step-by-step Trace (Log):** Crucial. What was the prompt for each step? LLM's response? Feedback generated? This detailed log is invaluable for understanding the agent's "thought" process and debugging.

Iterative Coding Example

Implementing feedback loops allows LLM-powered systems and AI agents to iteratively refine outputs. Feedback (from self-critique, external tools, validation checks, user input) is incorporated into subsequent prompts. Monitoring is essential to track progress, debug, and optimize. The code generation example shows how an AI can identify and correct errors autonomously.

12. Implementing LLM Feedback Loops with Python

13. Project : AgentsVille Trip Planner : A Multi-agent Travel Assistant System